

Final Project Report: Intelligent Behavioral Clone Controller

Course: MCTA 4362 Machine Learning

Project: Design of Intelligent Controllers for DC Motor Applications

Submitted By: MUHAMMAD NAZHAN BIN MOHAMED NADZRI

Matric Number: 2313703

GitHub Repository: <https://github.com/nazzhannn/miniprojectML>

1. Introduction

1.1 Problem Statement

Classical PID control systems, while industry standards for linear and stationary environments, encounter significant challenges when systems exhibit nonlinearities or time-varying disturbances. This project explores Machine Learning (ML) as an adaptive alternative, specifically investigating Supervised Learning to create an "Intelligent Controller" that functions as a behavioral clone of a baseline PID loop.

1.2 Project Objectives

- To model a second-order DC motor plant using discrete-time state-space representation.
- To implement a Multi-Layer Perceptron (MLP) Neural Network for behavioral cloning.
- To evaluate performance against a classical PID controller under noise and physical disturbances.
- To provide an interactive analysis terminal via a custom GUI.

2. Methodology

2.1 Plant Modeling and Discretization

The DC motor is modeled via its transfer function $G(s)$, derived from the interaction between armature inductance/resistance and rotor inertia/friction. The continuous state-space matrices were discretized using the Bilinear transformation to facilitate digital simulation.

2.2 Supervised Behavioral Cloning

The core methodology relies on the "Teacher-Student" paradigm:

1. Data Generation: A traditional PID controller was used as the "Expert Teacher" to generate 600 operational data points across six reference setpoints (10–60 rad/s).
2. Feature Engineering: The input vector consists of three critical control metrics: [e, integral e, de/dt].
3. ANN Architecture: The "Student" is an MLP Regressor (hidden layer sizes=(64, 32)) utilizing ReLU activation and the Adam optimizer to find the optimal mapping of system error to control voltage.

3. Results and Performance Analysis

3.1 Comparative Stress Testing

The system was stress-tested across three scenarios within the GUI terminal:

1. Ideal Tracking: Testing the model's interpolation accuracy.
2. Sensor Noise ($\sigma=0.5$): Testing the robustness of the ANN to high-frequency interference.
3. Sudden Load Disturbance: Testing transient response during an unexpected performance drop at $t=2.0$.

3.2 Performance Metrics (Summary)

The following table summarizes the average comparative results across the tested setpoints:

Performance Metric	Classical PID	Machine Learning (ANN)
Mean Squared Error (MSE)	20.0287	21.1824
Rise Time	10.64%	10.47%
Overshoot	0.150s	0.150s
Settling Time	0.600s	0.400s

4. Discussion

4.1 Intelligence vs. Memorization

The ML model demonstrates Generalization by successfully calculating control inputs for speeds not explicitly trained (e.g., 25 rad/s). The intelligence of the system stems from the ANN's ability to construct a smooth, non-linear mapping (a control surface) between error metrics and voltage, effectively re-discovering the physics of the motor through pattern recognition rather than explicit programming.

4.2 Limitations and Trade-offs

The data shows that the ANN performance fluctuates significantly under high noise compared to the PID. This is because the PID uses integral math to filter high-frequency noise, whereas the ANN trained on clean Expert data while attempts to faithfully track the noisy signal. This highlights a clear path for future work that is Physics-Informed Machine Learning (PIML) or the addition of a digital noise-filtering layer.

5. Conclusion

This project successfully achieved the implementation of a functional ML-based behavioral clone controller. The performance analysis confirms that while Supervised Learning is highly effective for behavioral imitation, robust control in noisy environments requires either noisy data augmentation or a transition to Reinforcement Learning frameworks, where the controller can learn to optimize its own policy while interacting with noisy, real-world disturbances.

6. Individual Contribution

Solo Contribution: As the sole developer, I performed all tasks: physical plant modeling, derivation of discretization matrices, configuration of the PID baseline, training of the MLPRegressor, design of the interactive Gradio GUI terminal, and final comparative performance analysis.

Appendices:

